

RAG + Pydantic AI Agents with Ollama for Offline Use

A Complete Developer Tutorial

Django · pgvector · nomic-embed-text · llama3.2

Why Ollama Is Killing It Right Now

Ollama lets you run powerful LLMs (like Llama 3.2, Mistral, Phi-4) entirely on your own machine — no API key, no internet, no cost per query, no data leaving your system.

Before Ollama existed, running local LLMs was a nightmare — CUDA drivers, model format conversions, memory management, custom inference code. Ollama changed all that with a single install and a Docker-like CLI. It became the 'npm install' moment for local AI.

The 5 Reasons Developers Love Ollama

1. It's Dead Simple

One command to install, one command to run a model:

```
ollama run llama3.2
```

That's it. No virtual envs, no CUDA setup, no model downloads from obscure links. Ollama handles everything — model download, quantization, inference engine — in a single binary.

2. Privacy & GDPR Compliance

This is huge in Europe, and especially relevant for your work. When you use OpenAI or Claude APIs, your data leaves your server. With Ollama:

- **Zero data sent to external servers**
- **Medical, legal, financial documents stay local**
- **GDPR compliance is trivially satisfied**
- **Clients with strict data policies can use your app**

Your competitive advantage:

SMBSHield / Nia RAG Intelligence running on Ollama can pitch to Dutch law firms, hospitals, and banks that GPT-based solutions cannot touch. 'Your data never leaves your premises' is a €100k contract conversation opener.

3. It Has an OpenAI-Compatible API

Ollama exposes an HTTP API at localhost:11434 that mirrors OpenAI's format. This means:

- **Any library that works with OpenAI also works with Ollama**
- **Pydantic AI supports Ollama natively as a provider**
- **Swap between GPT-4 and Llama3 by changing ONE line**

```
# OpenAI                                     #  
Ollama  
model='openai:gpt-4o'  
model='ollama:llama3.2'
```

4. Embedding Models Are Included

Ollama ships with embedding models like nomic-embed-text, which is what makes RAG possible offline. You get:

- **nomic-embed-text — 768 dimensions, fast, excellent quality**
- **mxbai-embed-large — 1024 dimensions, higher accuracy**
- **all-minilm — Tiny and fast, great for testing**

```
ollama pull nomic-embed-text
```

Same embeddings, zero cost, runs offline. This is the critical piece that enables offline RAG.

5. Model Library Keeps Growing

The Ollama model library now includes every major open model:

llama3.2:3b

Fast responses, low RAM

4 GB

llama3.2:8b

General use, balanced

8 GB

mistral:7b

Code + analysis

8 GB

phi4:14b

Reasoning, complex tasks

16 GB

deepseek-r1:8b

Step-by-step reasoning

8 GB

qwen2.5:7b

Multilingual (great for NL)

8 GB

For RAG specifically: llama3.2:8b is the sweet spot. Small

enough to run on a laptop, smart enough to answer complex questions from retrieved context.

Pydantic AI + Ollama: The Dream Team

Pydantic AI is your agent framework. Ollama is your LLM provider. Together they give you type-safe, offline, production-ready AI agents with zero API costs.

Why This Combo Works So Well

- **Pydantic AI gives you structured outputs, tool calling, dependency injection**
- **Ollama gives you offline inference with no rate limits**
- **pgvector gives you semantic search inside Postgres you already have**
- **Django ties it all together in your beloved batteries-included framework**

The key insight: Pydantic AI treats Ollama as just another model provider. The agent code is identical — only the model string changes. This means you can dev locally on Ollama and deploy with Claude or GPT-4 in production with zero code changes.

The Full RAG Pipeline with Ollama

This is the complete implementation. We'll build a RAG system for Professor Shield (OWASP cybersecurity tutor) step by step.

Step 1: Install Ollama and Pull Models

Install Ollama on your machine (Mac/Linux/Windows):

```
# macOS
brew install ollama

# Linux / WSL
curl -fsSL https://ollama.com/
install.sh | sh
```

Pull the models you need:

```
# The LLM (for answering questions)
ollama pull llama3.2

# The embedding model (for RAG)
ollama pull nomic-embed-text

# Verify they're running
ollama list
```

Check it works:

Run: `ollama run llama3.2` — you should get an interactive chat.
Type `/bye` to exit. If that works, you're ready.

Step 2: Django + pgvector Setup

Install required Python packages:

```
pip install pydantic-ai[ollama]  
pgvector psycopg2-binary django
```

Enable pgvector in Postgres:

```
-- In your Postgres DB (run once)  
CREATE EXTENSION IF NOT EXISTS vector;
```

Create the Django model for document chunks:

```
# models.py  
from django.db import models  
from pgvector.django import VectorField  
  
class DocumentChunk(models.Model):  
    content = models.TextField()  
    source_file =  
models.CharField(max_length=255)  
    embedding =  
VectorField(dimensions=768) # nomic-  
embed-text dim  
  
    class Meta:
```

```

        # HNSW index for fast
similarity search
        class Meta:
            indexes = [
                HnswIndex(

name='chunk_embedding_idx',
                fields=['embedding'],
                m=16,
ef_construction=64,

opclasses=['vector_cosine_ops']
                )
            ]

python manage.py makemigrations &&
python manage.py migrate

```

Step 3: Load & Chunk Documents

Get your OWASP data (one-time, manual):

```

git clone https://github.com/OWASP/
CheatSheetSeries.git

```

Then process the markdown files:

```

# scripts/ingest_owasp.py

import os

```



```

from langchain.text_splitter import
RecursiveCharacterTextSplitter

def load_owasp_docs(folder_path):
    docs = []
    for filename in
os.listdir(folder_path):
        if filename.endswith('.md'):
            with open(f'{folder_path}/
{filename}', 'r') as f:
                docs.append({'content':
f.read(), 'source': filename})
    return docs

def chunk_documents(docs):
    splitter =
RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200,    # Overlap
keeps context across boundaries
        separators=['\n\n', '\n', '. ',
' ', '']
    )
    chunks = []
    for doc in docs:
        for text in
splitter.split_text(doc['content']):
            chunks.append({'content':
text, 'source': doc['source']})
    return chunks

```

Step 4: Generate Embeddings with Ollama

Convert text chunks to vector embeddings using Ollama locally
— no API call, no cost:

```
# Add to scripts/ingest_owasp.py

import ollama

def embed_chunks(chunks):
    for i, chunk in enumerate(chunks):
        response = ollama.embeddings(
            model='nomic-embed-text',
            prompt=chunk['content']
        )
        chunk['embedding'] =
response['embedding'] # 768 floats

        if i % 50 == 0:
            print(f'Embedded {i}/
{len(chunks)} chunks...')

    return chunks
```

Why nomic-embed-text? It produces 768-dimensional embeddings, is free, runs fast on CPU, and matches or beats OpenAI's text-embedding-ada-002 on retrieval benchmarks. Zero cost, offline, GDPR-clean.

Step 5: Store in pgvector

```
# Add to scripts/ingest_owasp.py

import django
import os
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'smbshield.settings')
django.setup()

from your_app.models import
DocumentChunk

def store_chunks(chunks):
    objs = [
        DocumentChunk(
            content=c['content'],
            source_file=c['source'],
            embedding=c['embedding']
        )
        for c in chunks
    ]

    DocumentChunk.objects.bulk_create(objs,
batch_size=100)
    print(f'Stored {len(objs)} chunks
in pgvector')

# Run it once
if __name__ == '__main__':
    docs =
load_owasp_docs('CheatSheetSeries/
cheatsheets')
    chunks = chunk_documents(docs)
```

```
chunks = embed_chunks(chunks)
store_chunks(chunks)
```

Run it:

python scripts/ingest_owasp.py — This is a ONE-TIME process. After this, all your OWASP knowledge is in Postgres, ready for instant retrieval.

Step 6: RAG Retrieval Function

The retrieval function that finds the most relevant chunks for any question:

```
# your_app/rag_utils.py

import ollama
from pgvector.django import
CosineDistance
from .models import DocumentChunk

def retrieve_context(question: str,
top_k: int = 3) -> list[dict]:
    '''Find the most relevant OWASP
chunks for a question'''

    # 1. Embed the question with the
SAME model used for indexing
    q_embedding = ollama.embeddings(
        model='nomic-embed-text',
        prompt=question
```

```

) ['embedding']

# 2. Find closest chunks by cosine
similarity
results = (
    DocumentChunk.objects
        .annotate(distance=CosineDistance('embedding', q_embedding))
        .order_by('distance')
        [:top_k]
)

return [
    {'content': r.content,
 'source': r.source_file, 'score': 1 -
r.distance}
    for r in results
]

```

Step 7: Pydantic AI Agent with Ollama

This is where Pydantic AI and Ollama join forces. The agent uses Ollama as its LLM provider and calls your retrieval function as a tool:

```

# agents/professor_shield.py

from pydantic_ai import Agent,
RunContext
from pydantic import BaseModel, Field
from typing import Optional

```

```

# --- Structured Output (Pydantic
validates this) ---

class SecurityAnswer(BaseModel):
    answer: str
    sources: list[str] =
Field(default_factory=list)
    confidence: float =
Field(ge=0.0, le=1.0)
    follow_up_questions: list[str] =
Field(default_factory=list,
max_length=3)

# --- Agent Dependencies ---

class ProfessorDeps(BaseModel):
    user_question: str

# --- The Agent (Ollama as provider)
---

SYSTEM_PROMPT = '''
You are Professor Shield, an expert
cybersecurity educator.
Answer ONLY using the OWASP
documentation provided by your tool.
If the context doesn't contain the
answer, say so honestly.
Always cite your sources.
'''

```

```

professor = Agent(
    'ollama:llama3.2',          # <--
    Ollama, fully offline!
    deps_type=ProfessorDeps,
    result_type=SecurityAnswer, # <--
    Pydantic validates the response
    system_prompt=SYSTEM_PROMPT
)

# --- Tool: This is what makes it RAG
---

@professor.tool
def retrieve_owasp_context(ctx:
    RunContext[ProfessorDeps]) -> str:
    '''Retrieve relevant OWASP
    documentation for the question'''
    from your_app.rag_utils import
    retrieve_context

    chunks =
    retrieve_context(ctx.deps.user_question
    , top_k=3)

    if not chunks:
        return 'No relevant
        documentation found.'

    return '\n\n'.join(
        f'[Source: {c["source"]}]\n{c["content"]}'
        for c in chunks
    )

```

```

    )

# --- Usage ---

async def ask_professor(question: str)
-> SecurityAnswer:
    result = await professor.run(
        'Answer this cybersecurity
question',

deps=ProfessorDeps(user_question=questi
on)
    )
    return result.data # Fully
validated SecurityAnswer

```

Step 8: Django View

Wire it into your Django app:

```

# views.py

from django.http import JsonResponse
from django.views.decorators.http
import require_POST
import asyncio, json
from .agents.professor_shield import
ask_professor

@require_POST
def chat_with_professor(request):

```



```
data      = json.loads(request.body)
question = data.get('message',
'').strip()

if not question:
    return JsonResponse({'error':
'No question provided'}, status=400)

answer =
asyncio.run(ask_professor(question))

return JsonResponse({
    'answer':
answer.answer,
    'sources':
answer.sources,
    'confidence':
answer.confidence,
    'follow_up_questions':
answer.follow_up_questions,
})
```

Ollama vs OpenAI for RAG: When to Use What

Cost

Free after hardware

\$0.002–\$0.06 per 1K tokens

Privacy

100% local, GDPR-clean

Data sent to OpenAI

Speed

Depends on your GPU/CPU

Fast, consistent

Model quality

Llama3.2 $\approx$ GPT-3.5 level

GPT-4o is state of art

Reliability

Always available offline

Requires internet

Setup

One CLI install

API key, billing setup

Best for

Dev, GDPR-sensitive clients

Production at scale

The Smart Strategy: Both

Use Ollama for local development (fast iterations, no API costs) and provide a config switch to deploy with Claude or GPT-4 in production. Pydantic AI makes this trivial — one line change.

```
# config.py / .env  
# Development
```

```
AI_MODEL = 'ollama:llama3.2'

# Production (flip this in Railway/prod
env)
AI_MODEL = 'anthropic:claude-
sonnet-4-20250514'

# agents/professor_shield.py
from django.conf import settings

professor = Agent(
    settings.AI_MODEL,    # One setting,
    works everywhere
    ...
)
```

Complete Pipeline Summary

Here's the full picture of what you've built:

Ingest (once)

Download OWASP docs

git clone / manual

Chunk

Split into 1000-char pieces

RecursiveCharacterTextSplitter

Embed

Convert text to 768 vectors

Ollama: nomic-embed-text

Store

Save vectors + text to DB

pgvector + Django ORM

Query

User asks a question

Django view / HTMX

Retrieve

Find top-3 relevant chunks

cosine similarity query

Answer

Agent reads context + answers

Pydantic AI + Ollama llama3.2

Validate

Structured response checked

Pydantic SecurityAnswer

Return

Answer + sources + confidence

JSON to frontend

What you've built:

A fully offline, GDPR-compliant, hallucination-resistant AI tutor that only answers from authoritative OWASP documentation. Zero API costs in development. One config change to deploy with Claude or GPT-4 in production.

This is interview gold. The answer: 'I built a grounded RAG system with Ollama for offline inference, pgvector for semantic search, and Pydantic AI for structured validated outputs. It runs 100% locally, is GDPR-compliant, and I can swap the LLM provider in one line.'